

Universidade da Beira Interior

Departamento de Informática



Departamento de
Informática

Renderização Volumétrica de Nuvens em Tempo Real

Relatório de Projeto — Unidade Curricular de Computação Gráfica

Elaborado por:

Gustavo Dias

Orientador:

Professor/a Doutor/a [Nome do Orientador]

25 de junho de 2026

Agradecimentos

A conclusão deste trabalho não teria sido possível sem o apoio e orientação recebidos ao longo do semestre. Agradeço ao corpo docente da Unidade Curricular de Computação Gráfica pela transmissão de conhecimentos fundamentais em renderização e processamento gráfico, que constituíram a base sobre a qual este projeto foi desenvolvido.

Agradeço também à comunidade de investigação que publicou os trabalhos sobre renderização volumétrica em tempo real, em especial à equipa da Guerrilla Games cujos artigos técnicos serviram de principal referência para este projeto.

Conteúdo

Lista de Figuras

Lista de Tabelas

Acrónimos

Capítulo

1

Introdução

1.1 Enquadramento

A renderização de fenómenos atmosféricos em tempo real é um dos desafios mais exigentes da computação gráfica moderna. As nuvens, em particular, são estruturas volumétricas de geometria indefinida, iluminação complexa e dinâmica constante, tornando a sua simulação fiel especialmente difícil de obter a taxas de imagem interativas.

Este projeto, desenvolvido no âmbito da unidade curricular de Computação Gráfica da **UBI! (UBI!)**, consiste na implementação de um renderizador de nuvens volumétricas em tempo real. A aplicação produz nuvens procedurais tridimensionais, iluminadas de forma fisicamente plausível, animadas pelo vento e controláveis interativamente pelo utilizador.

A técnica central é o *ray marching* volumétrico: para cada pixel do ecrã, um raio é lançado a partir da câmara e atravessa uma camada de nuvens definida por duas altitudes. Ao longo do percurso, amostras de ruído tridimensional determinam a densidade local, e um segundo conjunto de raios calcula a iluminação proveniente do sol, aplicando a lei de Beer-Lambert ? e a função de fase de Henyey-Greenstein ?.

1.2 Motivação

A renderização de nuvens em tempo real ganhou grande relevância na última década com a proliferação de simuladores de voo, videojogos de mundo aberto e motores de renderização em tempo real. Títulos como *Horizon Zero Dawn* da Guerrilla Games ? ou o *Microsoft Flight Simulator* demonstraram que é possí-

vel obter nuvens de grande qualidade visual em hardware de consumo corrente, motivando a exploração académica destes algoritmos.

A motivação para este projeto residiu precisamente na curiosidade de implementar, do zero e com controlo total sobre o pipeline, os algoritmos descritos na literatura técnica de computação gráfica, compreendendo em detalhe cada componente: geração de ruído, *ray marching*, modelo de iluminação, acumulação temporal e composição final.

1.3 Objetivos

Os objetivos definidos para este projeto foram:

1. Implementar um gerador de ruído 3D procedural (Perlin, Worley e Perlin-Worley) para definir a forma e os detalhes das nuvens;
2. Desenvolver um shader de *ray marching* que percorra uma camada atmosférica e calcule a densidade volumétrica das nuvens;
3. Incorporar um modelo de iluminação fisicamente plausível com dispersão de luz (*scattering*), absorção (Beer-Lambert) e efeito *powder*;
4. Implementar acumulação temporal (**TAA! (TAA!)**) para reduzir o ruído inerente ao *ray marching* estocástico;
5. Criar uma interface gráfica interativa (via ImGui) que permita ajustar em tempo real os parâmetros das nuvens, iluminação e câmara;
6. Aplicar *tonemapping* **ACES! (ACES!)** e correção gamma para converter o resultado HDR em sRGB exibível.

1.4 Organização do Documento

O presente documento encontra-se estruturado da seguinte forma:

1. O primeiro capítulo — **Introdução** — apresenta o projeto, o seu enquadramento no contexto académico, a motivação para a escolha do tema, os objetivos traçados e a organização deste relatório.
2. O segundo capítulo — **Estado da Arte** — revê os principais trabalhos e técnicas existentes na área da renderização volumétrica de nuvens, situando este projeto no contexto da investigação atual.

3. O terceiro capítulo — **Tecnologias e Ferramentas Utilizadas** — descreve as bibliotecas, *APIs* e linguagens que constituem a base tecnológica da implementação.
4. O quarto capítulo — **Implementação** — detalha as escolhas de arquitetura e os algoritmos implementados, desde a geração de ruído até à composição final com *tonemapping*.
5. O quinto capítulo — **Conclusões e Trabalho Futuro** — sintetiza as conclusões do projeto e aponta extensões e melhorias que poderiam ser realizadas futuramente.

Capítulo

2

Estado da Arte

2.1 Introdução

A renderização de nuvens é um problema estudado desde os primórdios da computação gráfica. Ao longo das décadas, as abordagens foram evoluindo de representações geométricas simplificadas para modelos físicos cada vez mais rigorosos. Este capítulo revê as contribuições mais relevantes, desde os modelos clássicos de volumes participativos até às técnicas modernas de *ray marching* em tempo real, situando as escolhas de implementação deste projeto no contexto da investigação atual.

A secção ?? introduz a teoria de renderização de meios participativos. A secção ?? aborda os algoritmos de ruído procedural usados para modelar a forma das nuvens. A secção ?? descreve os modelos de iluminação aplicáveis a volumes. A secção ?? apresenta as técnicas modernas de renderização em tempo real. Por fim, a secção ?? resume as conclusões.

2.2 Renderização de Meios Participativos

Um meio participativo é qualquer substância que interage com a luz ao longo do seu percurso, em contraste com superfícies opacas ou transparentes ?. As nuvens são compostas por microgotículas de água que absorvem, emitem e dispersam a luz — um fenómeno descrito pela equação de transferência radiativa (*Radiative Transfer Equation*, RTE):

$$\frac{dL}{ds} = -(\sigma_a + \sigma_s) L + \sigma_a L_e + \sigma_s \int_{4\pi} p(\omega, \omega') L(\omega') d\omega' \quad (2.1)$$

onde L é a radiância, σ_a é o coeficiente de absorção, σ_s o de dispersão, L_e a emissão, e $p(\omega, \omega')$ a função de fase que descreve como a luz é redistribuída entre as direções ω' e ω ?.

Na prática, a solução analítica da equação (??) é intratável. As implementações em tempo real recorrem a simplificações: extinção única (sem dispersão múltipla explícita), lei de Beer-Lambert para modelar a transmitância, e uma função de fase analítica para o termo de dispersão simples.

A transmitância ao longo de um segmento de comprimento d com densidade ρ é dada pela lei de Beer-Lambert:

$$T(d) = e^{-\sigma_a \cdot \rho \cdot d} \quad (2.2)$$

2.3 Ruído Procedural para Modelação de Nuvens

2.3.1 Ruído de Perlin

Ken Perlin introduziu em 1985 o algoritmo de ruído gradiente que viria a ter o seu nome ?. O ruído de Perlin é contínuo, tem frequência fundamental bem definida, e a sua aparência orgânica e natural resulta da interpolação suave entre gradientes aleatórios numa grelha. Através da técnica de *Fractional Brownian Motion* (FBM! (FBM!)) — soma de oitavas com frequência crescente e amplitude decrescente — obtém-se um ruído com detalhes a várias escalas, adequado para modelar estruturas naturais como nuvens, terreno e fumo.

2.3.2 Ruído de Worley

Steven Worley propôs em 1996 um modelo de ruído baseado em distâncias a pontos de células ?. O espaço é dividido numa grelha e cada célula contém um ponto aleatório; o valor em qualquer posição é a distância ao ponto mais próximo. Este ruído, também designado *cellular noise* ou *Voronoi noise*, produz padrões semelhantes a bolhas ou células, ideais para modelar a estrutura interior esponjosa das nuvens cumulus.

2.3.3 Combinação Perlin-Worley

Andrew Schneider, na sua palestra técnica sobre *Horizon Zero Dawn* ?, popularizou a combinação dos dois ruídos. O ruído *Perlin-Worley* combina o perfil de larga escala do Perlin (FBM) com a estrutura celular do Worley, produzindo formas de nuvem com aspeto de “couve-flor” característico dos cumulus reais. A textura 3D de forma (*shape noise*) de 128^3 é codificada em 4 canais: o canal

R contém o Perlin-Worley, e os canais G, B e A contêm Worley a frequências crescentes para enriquecer o FBM em espaço de textura.

2.4 Modelos de Iluminação Volumétrica

2.4.1 Função de Fase de Henyey-Greenstein

Henyey e Greenstein propuseram em 1941 uma função analítica para descrever a dispersão de luz em meios participativos ?:

$$p_{HG}(\cos \theta, g) = \frac{1 - g^2}{4\pi (1 + g^2 - 2g \cos \theta)^{3/2}} \quad (2.3)$$

onde θ é o ângulo entre a direção de incidência e a direção de saída, e $g \in (-1, 1)$ é o parâmetro de assimetria. Para $g > 0$ a dispersão é predominantemente para a frente (*forward scattering*), criando o halo luminoso visível nas nuvens retroiluminadas pelo sol. Para $g < 0$ predomina a retrodispersão. Em renderização de nuvens, é comum combinar dois lobos (um para a frente e outro para trás) numa função de fase dupla, aproximando o fenómeno *silver lining*.

2.4.2 Efeito Powder

O efeito *powder* (ou *dark-edge*), também descrito por Schneider ?, é uma aproximação ao múltiplo espalhamento que produz as bordas escuras visíveis na base das nuvens cumulus. É obtido modulando a transmitância calculada por Beer-Lambert com um fator adicional que depende da densidade acumulada ao longo do raio de luz, tornando as zonas de maior densidade aparentemente mais escuras quando vistas de determinados ângulos.

2.5 Renderização em Tempo Real

2.5.1 Ray Marching Volumétrico

O *ray marching* é uma técnica de renderização que percorre um raio em passos discretos, amostrando a função de densidade ao longo do percurso ?. A acumulação do resultado é feita através da integração numérica de Riemann da equação de transferência radiativa simplificada. O principal desafio é o compromisso entre qualidade e desempenho: passos demasiado grandes produzem artefactos, enquanto passos pequenos tornam a renderização demasiado lenta para uso em tempo real.

2.5.2 Acumulação Temporal

A **TAA!** é uma técnica que distribui o custo computacional de uma imagem por múltiplos fotogramas consecutivos ?. No contexto de *ray marching*, o *jitter* introduzido no ponto de partida de cada raio (usando o padrão **IGN! (IGN!)**) garante que diferentes regiões do volume são amostradas em fotogramas distintos. A mistura (*blend*) com o fotograma anterior acumula a informação ao longo do tempo, convergindo para uma imagem mais suave sem custo adicional por fotograma. O desafio reside na gestão de *ghosting* quando a câmara se move: é necessário aumentar o peso do fotograma atual em caso de movimento.

2.5.3 Tonemapping ACES

Os valores de radiância calculados durante o *ray marching* são lineares e de intervalo alargado (**HDR! (HDR!)**). Para exibição num monitor sRGB é necessário comprimir este intervalo dinâmico. O *tonemapping ACES!* (*Academy Color Encoding System*) ? aplica uma curva S que preserva detalhes tanto nas zonas claras como nas escuras, tornando a imagem final mais equilibrada e esteticamente agradável.

2.6 Conclusões

A literatura existente estabelece claramente os componentes necessários para uma implementação eficiente de nuvens volumétricas em tempo real: ruído Perlin-Worley para a modelação de forma, *ray marching* com *jitter* para amostragem estocástica, função de fase HG dupla para iluminação realista, e acumulação temporal para convergência da imagem. Este projeto adota estas técnicas, implementando-as sobre uma base **OpenGL! (OpenGL!)**/GLSL, conforme descrito no Capítulo ??.

Capítulo

3

Tecnologias e Ferramentas Utilizadas

3.1 Introdução

Este capítulo descreve as tecnologias seleccionadas para a implementação do renderizador. A escolha privilegiou ferramentas maduras, bem documentadas e adequadas ao desenvolvimento de aplicações gráficas em ambiente académico. São apresentadas as linguagens de programação, a **API!** (**API!**) gráfica, as bibliotecas de suporte e o sistema de construção (*build system*).

A secção ?? aborda as linguagens utilizadas. A secção ?? descreve a **API!** gráfica **OpenGL!** e o **GLSL!** (**GLSL!**). As secções ?? a ?? apresentam as bibliotecas auxiliares. Por fim, a secção ?? descreve o sistema de construção.

3.2 C++17

A aplicação foi escrita em C++ seguindo o standard C++17. O C++ é a linguagem de eleição em computação gráfica de alto desempenho pela sua combinação de controlo de baixo nível sobre a memória e **GPU!** (**GPU!**) com abstrações de alto nível como classes, templates e gestão automática de recursos (RAII). O C++17 introduz melhorias como `std::optional`, estruturas de ligação (*structured bindings*) e garantias de *copy elision* que foram usadas para tornar o código mais expressivo.

3.3 OpenGL 4.2 e GLSL 4.20

OpenGL! ? é a **API!** gráfica de plataforma cruzada (*cross-platform*) mais utilizada em ambiente acadêmico e de produção. A versão 4.2 (*Core Profile*) foi escolhida por ser a versão mais recente suportada pelo *driver* Mesa em ambiente **WSL!** (**WSL!**)/Linux com aceleração D3D12 (AMD Radeon), garantindo compatibilidade sem perder funcionalidades necessárias.

O **GLSL!** na versão 4.20 (#version 420 core) é a linguagem de programação dos shaders. Os shaders implementados incluem:

- `fullscreen.vert` — vertex shader partilhado por todos os passes, que processa um quad de ecrã completo em **NDC!** (**NDC!**);
- `clouds.frag` — fragment shader principal, responsável pelo *ray marching* volumétrico e acumulação temporal;
- `composite.frag` — shader de composição, que combina as nuvens com o céu procedural, aplica *tonemapping* ACES e correção gamma.

3.4 GLFW 3.4

GLFW! (**GLFW!**) ? é uma biblioteca de criação de janelas e gestão de input para **OpenGL!**. Abstrai as diferenças entre sistemas operativos (Windows, Linux, macOS) fornecendo uma **API!** uniforme para criação de contextos OpenGL, captura de eventos de teclado e rato, e sincronização com o monitor (*vsync*). A versão 3.4 foi obtida automaticamente via CMake FetchContent durante a compilação.

3.5 GLAD v2

GLAD! (**GLAD!**) ? é um carregador de extensões **OpenGL!**. Uma vez que os pontos de entrada das funções OpenGL não estão disponíveis estaticamente no sistema operativo, é necessário um carregador que os resolva em tempo de execução. O GLAD v2 gera código de carregamento específico para a versão OpenGL 4.2 Core Profile, eliminando a necessidade de chamar `wglGetProcAddress`/`glXGetProcAddress` manualmente.

3.6 GLM 1.0.1

GLM! (**GLM!**) ? é uma biblioteca *header-only* de matemática vetorial e matricial modelada sobre o **GLSL!**. Fornece tipos como `glm::vec3`, `glm::mat4` e

`glm::quat` com operações otimizadas. Os tipos GLM são diretamente compatíveis com as funções de upload de uniformes do OpenGL (`glUniform*`), eliminando conversões manuais. Operações como `glm::inverse()`, `glm::normalize()` e `glm::length()` são usadas extensivamente no código da câmara e do renderer.

3.7 Dear ImGui 1.90.9

Dear ImGui ? é uma biblioteca de interface gráfica do utilizador de modo imediato (*immediate mode GUI*). A cada fotograma, os elementos de UI (sliders, botões, caixas de cor) são declarados e desenhados sem estado persistente gerido pela biblioteca. O backend OpenGL 3 e o backend GLFW são incluídos diretamente nos ficheiros fonte da biblioteca, facilitando a integração. A UI do projeto é composta por painéis colapsáveis que controlam os parâmetros de nuvens, iluminação, vento e câmara descritos na Tabela ??.

Grupo	Parâmetro	Valor Padrão
Nuvens	Altitude mínima (m)	1500
	Altitude máxima (m)	4000
	Cobertura	0.50
	Densidade	0.25
Ruído	Escala shape	0.000065
	Escala detail	0.00035
	Força detail	0.20
Vento	Direção	(1, 0, 0.1)
	Velocidade (m/s)	40
Iluminação	Direção do sol	(0.5, 0.8, 0.2)
	Intensidade solar	2.5
	Absorção (luz)	0.40
	Absorção (câmara)	0.20
	Dark edge factor	0.60
HG Scattering	g_+ (forward)	0.85
	g_- (backward)	-0.25
	Mistura	0.50
Ray March	Passos primários	64
	Passos de luz	8

Tabela 3.1: Parâmetros configuráveis via ImGui e respetivos valores padrão.

3.8 CMake 3.20

O CMake é o sistema de construção utilizado. O ficheiro `CMakeLists.txt` declara todas as dependências via `FetchContent`, que as descarrega e compila automaticamente durante a configuração do projeto. Isto garante que a aplicação compila de forma determinista em qualquer máquina com C++17 e CMake 3.20, sem necessidade de instalar manualmente bibliotecas externas.

3.9 Conclusões

O conjunto de tecnologias escolhido — C++17, OpenGL 4.2, GLFW, GLAD, GLM e ImGui — constitui uma pilha tecnológica madura e bem documentada para desenvolvimento de renderizadores em tempo real. A gestão automática de dependências via CMake `FetchContent` simplifica a configuração do ambiente de desenvolvimento, permitindo focar o esforço na implementação dos algoritmos de renderização descritos no capítulo seguinte.

Capítulo

4

Implementação

4.1 Introdução

Este capítulo descreve em detalhe a implementação do renderizador de nuvens volumétricas. A aplicação é composta por cinco módulos principais: *Application* (ciclo de vida da janela e UI), *Camera* (câmara FPS livre), *NoiseGenerator* (geração das texturas 3D de ruído), *CloudRenderer* (gestão do pipeline de renderização) e os shaders GLSL (*clouds.frag* e *composite.frag*).

A secção ?? descreve a arquitetura geral. A secção ?? detalha a geração de ruído. A secção ?? explica o pipeline de renderização a dois passes. A secção ?? aborda o modelo de iluminação. A secção ?? trata da acumulação temporal. A secção ?? descreve a composição final. Por fim, a secção ?? explica a câmara interativa.

4.2 Arquitetura Geral

A aplicação segue um ciclo de renderização clássico, ilustrado na Tabela ?? . A cada fotograma são executados dois passes de renderização distintos: um passe de *ray marching* que acumula a cor e a transmitância das nuvens para um *framebuffer* intermédio, e um passe de composição que combina o resultado com o céu procedural e aplica *tonemapping*.

Passe	Shader	Saída
1	<code>clouds.frag</code>	Cor das nuvens (RGBA16F) + Transmitância (R16F) em FBO
2	<code>composite.frag</code>	Imagem final (RGBA8) no framebuffer padrão

Tabela 4.1: Passes de renderização por fotograma.

O passe 1 usa dois pares de **FBO! (FBO!)** em *ping-pong*: enquanto um é escrito no fotograma atual, o outro (do fotograma anterior) é lido para a acumulação temporal. O módulo `CloudRenderer` gere esta alternância automaticamente com a variável `currentBuf`.

4.3 Geração de Ruído Procedural

O módulo `NoiseGenerator` gera duas texturas 3D na **CPU! (CPU!)** durante a inicialização: a textura de forma (*shape noise*) de 128^3 voxels, e a textura de detalhe (*detail noise*) de 32^3 voxels. Ambas são codificadas em RGBA32F e carregadas para a **GPU!** com mipmapping trilinear.

4.3.1 Ruído de Perlin

O ruído de Perlin é implementado com a tabela de permutação original de Ken Perlin [?], com 256 entradas duplicadas para evitar operações de módulo no interior do loop. A função de interpolação é a curva suave de quinta ordem (*smoothstep* quintico):

$$f(t) = 6t^5 - 15t^4 + 10t^3 \quad (4.1)$$

O **FBM!** é calculado como soma de três oitavas com amplitudes decrescentes:

$$\text{FBM}(x, y, z) = 0.500 \cdot P(4x, 4y, 4z) + 0.350 \cdot P(8x, 8y, 8z) + 0.150 \cdot P(16x, 16y, 16z) \quad (4.2)$$

onde P denota o ruído de Perlin normalizado para $[0, 1]$.

4.3.2 Ruído de Worley

O ruído de Worley utiliza um hash de inteiros (XOR com constantes primas) para gerar pontos de célula deterministicamente, evitando o custo de um gerador de números aleatórios. A vizinhança $3 \times 3 \times 3$ de células é percorrida para cada voxel, calculando a distância euclidiana ao ponto mais próximo. O resultado é invertido ($1 - d$) para que as células produzam picos em vez de vales, facilitando a composição com o Perlin.

4.3.3 Combinação Perlin-Worley e Canais de Textura

A textura de forma de 128^3 armazena em quatro canais:

- **R:** Perlin-Worley — combinação de FBM Perlin (60%) com Worley invertido a freq. 4 (40%), produzindo o perfil de “couve-flor” característico das nuvens cumulus;
- **G:** Worley invertido, frequência 4;
- **B:** Worley invertido, frequência 8;
- **A:** Worley invertido, frequência 16.

A textura de detalhe de 32^3 armazena três oitavas de Worley (freq. 2, 4 e 8) nos canais R, G e B, usadas para erosão das bordas das nuvens.

4.4 Pipeline de Ray Marching

O shader `clouds.frag` implementa o *ray marching* volumétrico em cinco etapas por fragmento.

4.4.1 Reconstrução do Raio

Dado o fragmento com coordenadas UV $[0, 1]^2$, o ponto em **NDC!** é calculado como $\text{ndc} = 2 \cdot \text{uv} - 1$. O raio em espaço de câmara é obtido pela inversão da matriz de projeção, e transformado para espaço de mundo pela inversão da matriz de vista:

$$\mathbf{r}_d = \text{normalize} \left((M_{view}^{-1} \cdot M_{proj}^{-1} \cdot (\text{ndc}, -1, 0))_{xyz} \right) \quad (4.3)$$

4.4.2 Interseção com a Camada de Nuvens

A camada de nuvens é definida por dois planos horizontais: altitude mínima h_{bot} e altitude máxima h_{top} . O intervalo de marcha $[t_{start}, t_{end}]$ é determinado em função da posição vertical da câmara:

- **Câmara abaixo da camada** ($y < h_{bot}$): o raio entra no plano inferior e sai no plano superior;
- **Câmara acima da camada** ($y > h_{top}$): o raio entra no plano superior;
- **Câmara dentro da camada:** o ray march começa na posição da câmara ($t_{start} = 0$) e termina no primeiro plano intersectado; raios horizontais ($r_{dy} \approx 0$) marcham até à distância máxima configurável.

O passo de marcha é limitado a 150 m independentemente do comprimento total do percurso, evitando subamostragem a ângulos rasantes.

4.4.3 Amostragem de Densidade

Para cada posição $\mathbf{p} = \mathbf{r}_o + t \cdot \mathbf{r}_d$ ao longo do raio, a densidade das nuvens é calculada como:

1. **Perfil de altitude:** gradiente suave que anula a densidade na base e no topo da camada, produzindo nuvens com base plana e topo arredondado:

$$h_{grad} = \text{remap}(h_{norm}, 0, 0.15, 0, 1) \cdot \text{remap}(h_{norm}, 0.85, 1, 1, 0) \quad (4.4)$$

onde h_{norm} é a altitude normalizada $[0, 1]$ dentro da camada.

2. **Shape noise:** amostragem da textura 3D com deslocamento de vento, seguida de FBM em espaço de textura:

$$S_{FBM} = 0.625R + 0.250G + 0.100B + 0.025A \quad (4.5)$$

3. **Máscara de cobertura:** remapeia S_{FBM} de forma a que apenas valores acima de $(1 - \text{coverage})$ produzam nuvem, permitindo controlar a percentagem de céu encoberto.
4. **Erosão por detail noise:** o ruído de detalhe erode as bordas das nuvens de forma dependente da altitude, produzindo bordas mais suaves no topo e mais abruptas na base.

A função de remapeamento remap é definida como:

$$\text{remap}(v, lo, hi, newLo, newHi) = newLo + (newHi - newLo) \cdot \text{clamp}\left(\frac{v - lo}{hi - lo}, 0, 1\right) \quad (4.6)$$

4.4.4 Jitter por IGN

Para reduzir artefactos de amostragem, o ponto de partida de cada raio é perturbado por um valor pseudo-aleatório calculado com o padrão **IGN!** (*Interleaved Gradient Noise*):

$$\text{IGN}(\mathbf{p}) = \text{fract}(52.98 \cdot \text{fract}(\mathbf{p} \cdot (0.0671, 0.0058))) \quad (4.7)$$

O índice de fotograma multiplica o vetor de frequência, garantindo padrões diferentes em cada frame e convergência temporal com a **TAA!**.

4.5 Modelo de Iluminação

4.5.1 Light March

Para cada ponto p com densidade positiva, é realizado um segundo *ray march* na direção do sol, da posição p até ao topo da camada. A densidade acumulada D_{sun} determina a transmitância solar por Beer-Lambert (equação ??) com coeficiente σ_{light} :

$$T_{sun} = e^{-D_{sun} \cdot \sigma_{light}} \quad (4.8)$$

4.5.2 Efeito Powder

O efeito *powder* é calculado como:

$$T_{powder} = 1 - e^{-D_{sun} \cdot k_{dark} \cdot 2} \quad (4.9)$$

e misturado com T_{sun} em função do ângulo solar, introduzindo bordas escuras nas nuvens mais densas.

4.5.3 Função de Fase Dupla

A função de fase aplicada combina dois lobos de Henyey-Greenstein (equação ??):

$$p(\cos \theta) = (1 - \alpha) p_{HG}(\cos \theta, g^+) + \alpha p_{HG}(\cos \theta, g^-) \quad (4.10)$$

com valores padrão $g^+ = 0.85$ (dispersão frontal), $g^- = -0.25$ (retrodispersão) e $\alpha = 0.50$.

4.5.4 Integração de In-scattering

A contribuição luminosa de cada segmento de marcha é calculada pela integral analítica de Beer-Lambert, que evita divisão por σ_t quando a densidade é nula:

$$\Delta L = T_{acum} \cdot L_{total} \cdot (1 - e^{-\rho \cdot \sigma_{cam} \cdot \Delta t}) \quad (4.11)$$

onde T_{acum} é a transmitância acumulada até ao ponto atual, $L_{total} = L_{sun} + L_{amb}$ e Δt é o tamanho do passo. Este método é numericamente estável e não requer divisão por densidade.

4.6 Acumulação Temporal (TAA)

A acumulação temporal é implementada com dois pares de **FBO!** em regime de *ping-pong*. No final de cada fotograma, o resultado do passe de nuvens é misturado com o fotograma anterior:

$$C_n = (1 - \beta) C_{n-1} + \beta C_{atual} \quad (4.12)$$

O parâmetro β (blend temporal) adapta-se automaticamente ao movimento da câmara: em repouso assume o valor mínimo de 0.10 (favorece a história, convergindo rapidamente), e aumenta até 0.98 quando há translação ou rotação significativa (descarta a história para evitar *ghosting*):

$$\beta = \text{clamp}(0.10 + \Delta_{pos} \cdot 0.0005 + \Delta_{front} \cdot 500, 0.10, 0.98) \quad (4.13)$$

4.7 Composição e Tonemapping

O shader `composite.frag` compõe o resultado final em quatro etapas:

1. **Céu procedural:** gradiente zenith-horizonte com potência 0.4 sobre r_{dy} , mais névoa no horizonte;
2. **Disco solar:** *smoothstep* sobre o cosseno do ângulo ao sol, com halo suave;
3. **Composição:** as nuvens (cor + transmitância) são compostas sobre o fundo: $C_{final} = C_{nuvens} + C_{fundo} \cdot T$, onde $T = 1$ indica pixel totalmente transparente e $T = 0$ pixel totalmente opaco;
4. **Tonemapping ACES** seguido de correção gamma ($\gamma = 2.2$) para conversão para sRGB.

4.8 Câmara FPS

A câmara é do tipo FPS livre, inicializada a 2500 m de altitude (dentro da camada de nuvens). O movimento é controlado por teclado (WASD + Espaço/Ctrl) e o ângulo pela captura do rato (botão direito mantido). A câmara expõe as matrizes de vista e projeção que são invertidas no shader para a reconstrução dos raios (equação ??).

4.9 Conclusões

A implementação cobre todos os componentes enunciados nos objetivos: geração de ruído Perlin-Worley na **CPU!**, *ray marching* volumétrico com *jitter* IGN, modelo de iluminação Beer-Lambert com fase HG dupla e efeito *powder*, acumulação temporal adaptativa e composição com *tonemapping* ACES. O pipeline a dois passes mantém o código modular e facilita a manutenção e expansão futura.

Conclusões e Trabalho Futuro

5.1 Conclusões Principais

Este projeto demonstrou que é possível implementar um renderizador de nuvens volumétricas em tempo real de qualidade visual elevada, utilizando exclusivamente técnicas de domínio público e bibliotecas *open source*, sobre uma base **OpenGL! 4.2**.

Os objetivos definidos foram todos atingidos:

- A geração de ruído Perlin-Worley na **CPU!** produziu texturas 3D com a estrutura característica das nuvens cumulus, validando a abordagem de Schneider ? sem recorrer a compute shaders;
- O *ray marching* com *jitter* IGN e passo máximo de 150 m produziu imagens sem artefactos de amostragem visíveis a taxas de imagem interativas;
- O modelo de iluminação Beer-Lambert com função de fase HG dupla e efeito *powder* reproduz os fenómenos de retroiluminação (*silver lining*) e as bordas escuras típicas das nuvens cumulus pesadas;
- A acumulação temporal com blend adaptativo eliminou o ruído estocástico inerente ao *ray marching* com apenas 64 passos primários, permitindo obter imagens suaves sem aumentar o custo por fotograma;
- A interface ImGui permite explorar o espaço de parâmetros em tempo real, confirmando a influência de cada variável no resultado visual.

Do ponto de vista de aprendizagem, o projeto permitiu consolidar conhecimentos de **GLSL!**, álgebra linear aplicada a computação gráfica (matrizes de

vista/projeção inversas, transformações de espaço), física de volumes participativos e técnicas de anti-aliasing temporal.

5.2 Trabalho Futuro

Várias extensões poderiam melhorar significativamente a qualidade visual e o desempenho da aplicação:

1. **Geração de ruído na GPU via compute shaders:** o ficheiro `noise_shape.comp` e `noise_detail.comp` existentes no repositório mas ainda não integrados poderiam mover a geração de ruído para a **GPU!**, reduzindo o tempo de arranque de vários segundos (atualmente dominado pelo cálculo na **CPU!**) para milissegundos;
2. **Renderização a meia resolução com upscale:** renderizar o passe de *ray marching* a $\frac{1}{2}$ ou $\frac{1}{4}$ da resolução do ecrã e reconstruir a imagem final com *checkerboard rendering* ou DLSS/FSR reduziria o custo por fotograma proporcionalmente;
3. **Dispersão atmosférica fisicamente baseada:** substituir o gradiente de céu procedural por uma solução da equação de Rayleigh-Mie (como o modelo de Hillaire ?) produziria cores de céu e pôr do sol fisicamente corretos, dependentes da posição do sol;
4. **Sistema de meteorologia dinâmica:** adicionar um mapa de cobertura 2D com variação temporal permitiria simular frentes meteorológicas, células de tempestade e transições entre tempo limpo e encoberto;
5. **Múltiplo espalhamento aproximado:** a implementação atual aproxima o múltiplo espalhamento apenas com o efeito *powder*. Técnicas como a energia de múltiplos espalhamentos de Hillaire ? ou a aproximação de Wrenninge et al. produziriam nuvens com aspeto ainda mais realista em condições de luz difusa;
6. **Nímbos e cirros:** a camada atual modela apenas nuvens cumulus médias. Adicionar uma segunda camada de cirros finos a alta altitude (acima dos 6000 m) e células de nevoeiro/nimbostratus enriqueceria a diversidade visual da cena.